ITAI 1371 – Module 12 Lab

Individual Submission

Alexander Debusk

Ethics, Fairness, and Bias in Machine Learning

Module 12 Lab - Ethics, Fairness, and Bias in ML

Objective: To understand how machine learning models can inherit and amplify societal biases, how to measure this bias using fairness metrics, and to think critically about the ethical implications of deploying ML systems.

In this lab, you will train a model on a real-world dataset and audit it for fairness across different demographic groups.

Part 1: What is Algorithmic Bias?

Concept: Machine learning models learn from data. If the data reflects existing societal biases, the model will learn those biases. An "unbiased" algorithm trained on biased data will produce a biased model. This can lead to systems that are systematically unfair to certain groups of people.

Sources of Bias:

Historical Bias: The data reflects a world with historical injustices (e.g., past hiring data may show fewer women in leadership roles). Measurement Bias: The way we collect or measure data is flawed (e.g., using arrest records as a proxy for crime, which can be influenced by policing patterns). Representation Bias: The data underrepresents certain groups, so the model doesn't learn to perform well for them.

Problem: We will use the "Adult" dataset, which is used to predict whether an individual's income is greater than $50k/year. It contains sensitive attributes like sex and race, which we can use to audit our model for bias.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score

# Load the data
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult

columns = [
    'age', 'workclass', 'fnlwgt', 'education', 'education-num',
    'marital-status', 'occupation', 'relationship', 'race', 'sex',
    'capital-gain', 'capital-loss', 'hours-per-week',
    'native-country', 'income'
]
```

```
]

df = pd.read_csv(
    url,
    header=None,
    names=columns,
    sep=',\s*',
    engine='python',
    na_values='?'
)

# Data Cleaning
df.dropna(inplace=True)
df['income'] = df['income'].map({'<=50K': 0, '>50K': 1})

# Features and target
X = df.drop('income', axis=1)
y = df['income']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# Create a preprocessing pipeline
numeric_features = X.select_dtypes(include='number').columns
categorical_features = X.select_dtypes(exclude='number').columns

preprocessor = make_column_transformer(
    (StandardScaler(), numeric_features),
    (OneHotEncoder(handle_unknown='ignore'), categorical_features)
)

# Train a baseline model
model = make_pipeline(
    preprocessor,
    LogisticRegression(max_iter=1000)
)

model.fit(X_train, y_train)

# Evaluate
print(f"Overall model accuracy: {model.score(X_test, y_test):.2%}")
```

```
<>:23: SyntaxWarning: invalid escape sequence '\s'
<>:23: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_564/2379571722.py:23: SyntaxWarning: invalid escape sequence '
  sep=',\s*',
Overall model accuracy: 84.61%
```

Double-click (or enter) to edit

Part 2: Auditing the Model for Fairness

High overall accuracy can hide poor performance on specific subgroups. To properly evaluate the model, we need to audit its performance across sensitive attributes, such as sex.

Concept: Group Fairness

One common fairness goal is to ensure that the model performs equally well for different groups.

We can evaluate this by:

Calculating performance metrics (like accuracy) Separately for each subgroup (e.g., males vs females) Your Task Create a function that: Filters the dataset by a subgroup (e.g., sex = Male) Computes the model's accuracy on that subgroup Use the function to calculate: Accuracy for Males Accuracy for Females Compare the results: Does one group have higher accuracy? Is the difference significant?

Double-click (or enter) to edit

```
# --- ENTER YOUR CODE HERE ---

def get_subgroup_accuracy(model, X_test, y_test, subgroup_column, subgroup_v
    """Calculates accuracy for a specific subgroup of the test data."""

    # 1. Create a boolean mask to select the subgroup from X_test
    subgroup_mask = X_test[subgroup_column] == subgroup_value

    # 2. Select the subgroup data
    X_subgroup = X_test[subgroup_mask]
    y_subgroup = y_test[subgroup_mask]

    # 3. Calculate and return the model's score on this subgroup
    return model.score(X_subgroup, y_subgroup)


# 4. Calculate accuracy for males and females
acc_male = get_subgroup_accuracy(model, X_test, y_test, 'sex', 'Male')
acc_female = get_subgroup_accuracy(model, X_test, y_test, 'sex', 'Female')

print(f"Accuracy for Males: {acc_male:.2%}")
print(f"Accuracy for Females: {acc_female:.2%}")
```

```
Accuracy for Males: 81.20%
Accuracy for Females: 91.81%
```

Task 2: Deeper Dive with a Confusion Matrix

Accuracy alone doesn't tell the whole story. To better understand model behavior, we analyze the types of errors it makes for each group.

Your Task

Calculate and compare the False Positive Rate (FPR) and False Negative Rate (FNR) for males and females.

Definitions

False Positive Rate (FPR):

FPR =FP /(FP+TN)

The percentage of people who did NOT have high income but were incorrectly predicted to have high income.

False Negative Rate (FNR):

FNR= FN /(FN+TP)

The percentage of people who DID have high income but were incorrectly predicted to have low income.

```
from sklearn.metrics import confusion_matrix

def get_rates(model, X_test, y_test, subgroup_column, subgroup_value):
    # 1. Filter subgroup
    subgroup_mask = X_test[subgroup_column] == subgroup_value
    X_subgroup = X_test[subgroup_mask]
    y_subgroup = y_test[subgroup_mask]

    # 2. Predictions
    y_pred_subgroup = model.predict(X_subgroup)

    # 3. Confusion matrix
    tn, fp, fn, tp = confusion_matrix(y_subgroup, y_pred_subgroup).ravel()

    # 4. Rates
    fpr = fp / (fp + tn)
    fnr = fn / (fn + tp)

    return fpr, fnr
```

```
# --- ENTER YOUR CODE HERE ---

# 1. Calculate the rates for males and females
fpr_male, fnr_male = get_rates(model, X_test, y_test, 'sex', 'Male')
fpr_female, fnr_female = get_rates(model, X_test, y_test, 'sex', 'Female')

# 2. Print results
print(f"Male - False Positive Rate: {fpr_male:.2%}, False Negative Rate: {fn
print(f"Female - False Positive Rate: {fpr_female:.2%}, False Negative Rate:
```

```
Male - False Positive Rate: 10.26%, False Negative Rate: 37.80%
Female - False Positive Rate: 2.81%, False Negative Rate: 47.84%
```

Reflective Knowledge Check

1. Analyze Your Results

From the subgroup accuracy results:

Accuracy for Males: 81.20% Accuracy for Females: 91.81%

There is a noticeable difference in performance between the two groups, with a difference of 10.61 percentage points.

The model performs better for Females, as indicated by the higher accuracy. This suggests that the model may be biased toward that group, since it predicts outcomes more correctly for them.

2. Interpret the Errors

From the calculated rates:

Male FPR: 10.26%, Male FNR: 37.80% Female FPR: 2.81%, Female FNR: 47.84%

The model is more likely to make a False Positive error for Males, since that group has a higher FPR.

Real-world consequence (loan example): A False Positive means predicting high income when the person actually has low income.

This could lead to approving a loan for someone who may not be able to repay it Financial institutions could face higher default risk The model may unfairly favor one group in loan approvals

3. Justify a Decision (Hiring Scenario)

In a hiring context:

False Positive (FPR): Hiring someone predicted to be high-income potential when they are not False Negative (FNR): Rejecting someone who actually has high potential

More harmful error:

Typically, FNR is more harmful, because it denies qualified candidates opportunities This can lead to systematic exclusion of a group

Based on the results:

If one group has a higher FNR, they are unfairly being rejected more often If one group has a higher FPR, they are being unfairly favored

Decision:

I would not approve this model for deployment

Justification:

The model shows disparate impact, as error rates differ between groups This indicates potential bias, making it unsuitable for fair hiring decisions

4. Propose a Mitigation

Removing the 'sex' column will not fully eliminate bias.

Why?

Other features such as:

occupation education hours-per-week marital-status

may still be correlated with gender

This is called:

Proxy bias (indirect discrimination)

Conclusion:

Even after removing 'sex', the model may still learn gender-related patterns True fairness requires: Feature analysis Bias mitigation techniques (reweighting, fairness constraints, etc.)

ITAI 1371 – Module 12 Lab

Individual Submission

Shareefah Adeola

Ethics, Fairness, and Bias in Machine Learning

Module 12 Lab - Ethics, Fairness, and Bias in ML

Objective: To understand how machine learning models can inherit and amplify societal biases, how to measure this bias using fairness metrics, and to think critically about the ethical implications of deploying ML systems.

In this lab, you will train a model on a real-world dataset and audit it for fairness across different demographic groups.

Part 1: What is Algorithmic Bias?

Concept: Machine learning models learn from data. If the data reflects existing societal biases, the model will learn those biases. An "unbiased" algorithm trained on biased data will produce a biased model. This can lead to systems that are systematically unfair to certain groups of people.

Sources of Bias:

Historical Bias: The data reflects a world with historical injustices (e.g., past hiring data may show fewer women in leadership roles). Measurement Bias: The way we collect or measure data is flawed (e.g., using arrest records as a proxy for crime, which can be influenced by policing patterns). Representation Bias: The data underrepresents certain groups, so the model doesn't learn to perform well for them.

Problem: We will use the "Adult" dataset, which is used to predict whether an individual's income is greater than $50k/year. It contains sensitive attributes like sex and race, which we can use to audit our model for bias.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score

# Load the data
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data'

columns = [
    'age', 'workclass', 'fnlwgt', 'education', 'education-num',
    'marital-status', 'occupation', 'relationship', 'race', 'sex',
    'capital-gain', 'capital-loss', 'hours-per-week',
    'native-country', 'income'
]

df = pd.read_csv(
    url,
    header=None,
    names=columns,
    sep=',\s*',
    engine='python',
    na_values='?'
)

# Data Cleaning
df.dropna(inplace=True)
df['income'] = df['income'].map({'<=50K': 0, '>50K': 1})

# Features and target
X = df.drop('income', axis=1)
y = df['income']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# Create a preprocessing pipeline
numeric_features = X.select_dtypes(include='number').columns
categorical_features = X.select_dtypes(exclude='number').columns

preprocessor = make_column_transformer(
    (StandardScaler(), numeric_features),
    (OneHotEncoder(handle_unknown='ignore'), categorical_features)
)

# Train a baseline model
model = make_pipeline(
    preprocessor,
    LogisticRegression(max_iter=1000)
)

model.fit(X_train, y_train)
```

```
# Evaluate
print(f"Overall model accuracy: {model.score(X_test, y_test):.2%}")
```

```
<>:23: SyntaxWarning: invalid escape sequence '\s'
<>:23: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_638/2379571722.py:23: SyntaxWarning: invalid escape sequence '\s'
  sep=',\s*',
Overall model accuracy: 84.61%
```

Double-click (or enter) to edit

Part 2: Auditing the Model for Fairness

High overall accuracy can hide poor performance on specific subgroups. To properly evaluate the model, we need to audit its performance across sensitive attributes, such as sex.

Concept: Group Fairness

One common fairness goal is to ensure that the model performs equally well for different groups.

We can evaluate this by:

Calculating performance metrics (like accuracy) Separately for each subgroup (e.g., males vs females) Your Task Create a function that: Filters the dataset by a subgroup (e.g., sex = Male) Computes the model's accuracy on that subgroup Use the function to calculate: Accuracy for Males Accuracy for Females Compare the results: Does one group have higher accuracy? Is the difference significant?

Double-click (or enter) to edit

```
# --- ENTER YOUR CODE HERE ---

def get_subgroup_accuracy(model, X_test, y_test, subgroup_column, subgroup_value):
    """Calculates accuracy for a specific subgroup of the test data."""

    # 1. Create a boolean mask to select the subgroup from X_test
    subgroup_mask = X_test[subgroup_column] == subgroup_value

    # 2. Select the subgroup data
    X_subgroup = X_test[subgroup_mask]
    y_subgroup = y_test[subgroup_mask]

    # 3. Calculate and return the model's score on this subgroup
    return model.score(X_subgroup, y_subgroup)


# 4. Calculate accuracy for males and females
acc_male = get_subgroup_accuracy(model, X_test, y_test, 'sex', 'Male')
acc_female = get_subgroup_accuracy(model, X_test, y_test, 'sex', 'Female')

print(f"Accuracy for Males: {acc_male:.2%}")
print(f"Accuracy for Females: {acc_female:.2%}")
```

```
Accuracy for Males: 81.20%
Accuracy for Females: 91.81%
```

Task 2: Deeper Dive with a Confusion Matrix

Accuracy alone doesn't tell the whole story. To better understand model behavior, we analyze the types of errors it makes for each group.

Your Task

Calculate and compare the False Positive Rate (FPR) and False Negative Rate (FNR) for males and females.

Definitions

False Positive Rate (FPR):

FPR =FP /(FP+TN)

The percentage of people who did NOT have high income but were incorrectly predicted to have high income.

False Negative Rate (FNR):

FNR= FN /(FN+TP)

The percentage of people who DID have high income but were incorrectly predicted to have low income.

```
from sklearn.metrics import confusion_matrix

def get_rates(model, X_test, y_test, subgroup_column, subgroup_value):
    # 1. Filter subgroup
    subgroup_mask = X_test[subgroup_column] == subgroup_value
    X_subgroup = X_test[subgroup_mask]
    y_subgroup = y_test[subgroup_mask]
```

```
        # 2. Predictions
        y_pred_subgroup = model.predict(X_subgroup)

        # 3. Confusion matrix
        tn, fp, fn, tp = confusion_matrix(y_subgroup, y_pred_subgroup).ravel()

        # 4. Rates
        fpr = fp / (fp + tn)
        fnr = fn / (fn + tp)

        return fpr, fnr

    S1$0
    # --- ENTER YOUR CODE HERE ---

    # 1. Calculate the rates for males and females
    fpr_male, fnr_male = get_rates(model, X_test, y_test, 'sex', 'Male')
    fpr_female, fnr_female = get_rates(model, X_test, y_test, 'sex', 'Female')

    # 2. Print results
    print(f"Male - False Positive Rate: {fpr_male:.2%}, False Negative Rate: {fnr_male:.2%}")
    print(f"Female - False Positive Rate: {fpr_female:.2%}, False Negative Rate: {fnr_female:.2%}")
```

```
Male - False Positive Rate: 10.26%, False Negative Rate: 37.80%
Female - False Positive Rate: 2.81%, False Negative Rate: 47.84%
```

Reflective Knowledge Check

Analyze Your Results From the subgroup accuracy results:

Accuracy for Males: 81.20% Accuracy for Females: 91.81%

There is an observable difference in performance between the two groups.

The model performs better for Females (91.81%), as indicated by the higher accuracy. This suggests a potential fairness concern because the model predicts outcomes more accurately for females than males, with a difference of 10.61 percentage points.

Interpret the Errors From the calculated rates:

Male FPR: 10.26% Male FNR: 37.80% Female FPR: 2.81% Female FNR: 47.84%

The model is more likely to make a False Positive error for Males, since males have a higher FPR (10.26%) compared to females (2.81%).

A False Positive means predicting high income when the person actually has low income. In a real-world loan scenario, this could lead to approving a loan for someone who may not be able to repay it. Financial institutions could consequently face higher default risks. The model may also create unfairness in loan approval decisions because its error rates differ between males and females.

Justify a Decision (Hiring Scenario) In a hiring context:

A False Positive (FPR) means hiring someone predicted to have high potential when they actually do not. A False Negative (FNR) means rejecting someone who actually has high potential.

More harmful error:

Typically, a False Negative is more harmful because it denies qualified candidates employment opportunities. This can lead to systematic exclusion of a group.

Based on the results:

Females have a higher FNR (47.84%) than males (37.80%). This means the model is more likely to incorrectly reject females who actually have high potential. Males have a higher FPR (10.26%) than females (2.81%), meaning males are more likely to be incorrectly classified as having high potential.

Decision:

I would NOT APPROVE this model for deployment in hiring.

Justification:

The model shows differences in error rates between males and females, indicating potential bias. In particular, the higher False Negative Rate for females could result in qualified female candidates being unfairly rejected. The model should undergo further fairness testing and bias mitigation before it is used in hiring decisions.

Propose a Mitigation Removing the 'sex' column will not fully eliminate bias.

Why?

Other features such as occupation, education, hours-per-week, and marital-status may still be correlated with gender. These variables may indirectly influence the model's predictions and reproduce existing gender-related patterns.

This is called Proxy Bias (Indirect Discrimination).

Conclusion:

Even after removing the 'sex' column, the model may still learn gender-related patterns through other features.

True fairness requires:

Feature analysis to identify variables that may introduce indirect bias. Bias mitigation techniques, such as reweighting and fairness constraints, to improve fairness across demographic groups.

ITAI 1371 – Module 12 Lab

Individual Submission

Yilin Leng

Ethics, Fairness, and Bias in Machine Learning

Module 12 Lab – Ethics, Fairness, and Bias in ML

Objective: To understand how machine learning models can inherit and amplify societal biases, how to measure this bias using fairness metrics, and to think critically about the ethical implications of deploying ML systems.

In this lab, you will train a model on a real-world dataset and audit it for fairness across different demographic groups.

Part 1: What is Algorithmic Bias?

Concept: Machine learning models learn from data. If the data reflects existing societal biases, the model will learn those biases. An "unbiased" algorithm trained on biased data will produce a biased model. This can lead to systems that are systematically unfair to certain groups of people.

Sources of Bias:

Historical Bias: The data reflects a world with historical injustices (e.g., past hiring data may show fewer women in leadership roles). Measurement Bias: The way we collect or measure data is flawed (e.g., using arrest records as a proxy for crime, which can be influenced by policing patterns). Representation Bias: The data underrepresents certain groups, so the model doesn't learn to perform well for them.

Problem: We will use the "Adult" dataset, which is used to predict whether an individual's income is greater than $50k/year. It contains sensitive attributes like sex and race, which we can use to audit our model for bias.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score

# Load the data
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data'

columns = [
    'age', 'workclass', 'fnlwgt', 'education', 'education-num',
    'marital-status', 'occupation', 'relationship', 'race', 'sex',
    'capital-gain', 'capital-loss', 'hours-per-week',
    'native-country', 'income'
]

df = pd.read_csv(
    url,
    header=None,
    names=columns,
    sep=',\s*',
    engine='python',
    na_values='?'
)

# Data Cleaning
df.dropna(inplace=True)
df['income'] = df['income'].map({'<=50K': 0, '>50K': 1})

# Features and target
```

```
X = df.drop('income', axis=1)
y = df['income']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# Create a preprocessing pipeline
numeric_features = X.select_dtypes(include='number').columns
categorical_features = X.select_dtypes(exclude='number').columns

preprocessor = make_column_transformer(
    (StandardScaler(), numeric_features),
    (OneHotEncoder(handle_unknown='ignore'), categorical_features)
)

# Train a baseline model
model = make_pipeline(
    preprocessor,
    LogisticRegression(max_iter=1000)
)

model.fit(X_train, y_train)

# Evaluate
print(f"Overall model accuracy: {model.score(X_test, y_test):.2%}")
```

```
<>:23: SyntaxWarning: invalid escape sequence '\s'
<>:23: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_687/2379571722.py:23: SyntaxWarning: invalid escape sequence '\s'
  sep=',\s*',
Overall model accuracy: 84.61%
```

Double-click (or enter) to edit

Part 2: Auditing the Model for Fairness

High overall accuracy can hide poor performance on specific subgroups. To properly evaluate the model, we need to audit its performance across sensitive attributes, such as sex.

Concept: Group Fairness

One common fairness goal is to ensure that the model performs equally well for different groups.

We can evaluate this by:

Calculating performance metrics (like accuracy) Separately for each subgroup (e.g., males vs females) Your Task Create a function that: Filters the dataset by a subgroup (e.g., sex = Male) Computes the model's accuracy on that subgroup Use the function to calculate: Accuracy for Males Accuracy for Females Compare the results: Does one group have higher accuracy? Is the difference significant?

Double-click (or enter) to edit

```
def get_subgroup_accuracy(model, X_test, y_test, subgroup_column, subgroup_value):
    """Calculates accuracy for a specific subgroup of the test data."""
    # 1. Create a boolean mask to select the subgroup from X_test
    subgroup_mask = X_test[subgroup_column] == subgroup_value

    # 2. Select the subgroup data
    X_subgroup = X_test[subgroup_mask]
    y_subgroup = y_test[subgroup_mask]

    # 3. Calculate and return the model's score on this subgroup
    return model.score(X_subgroup, y_subgroup)


# 4. Calculate accuracy for males and females
acc_male = get_subgroup_accuracy(model, X_test, y_test, 'sex', 'Male')
acc_female = get_subgroup_accuracy(model, X_test, y_test, 'sex', 'Female')

# 5. Print results and comparison
print(f"Accuracy for Males: {acc_male:.2%}")
print(f"Accuracy for Females: {acc_female:.2%}")

difference = abs(acc_male - acc_female)
print(f"Difference: {difference:.2%}")

if acc_male > acc_female:
    print("Males have higher accuracy.")
elif acc_female > acc_male:
    print("Females have higher accuracy.")
else:
    print("Both groups have the same accuracy.")
```

```
Accuracy for Males: 81.20%
Accuracy for Females: 91.81%
Difference: 10.61%
Females have higher accuracy.
```

Task 2: Deeper Dive with a Confusion Matrix

Accuracy alone doesn't tell the whole story. To better understand model behavior, we analyze the types of errors it makes for each group.

Your Task

Calculate and compare the False Positive Rate (FPR) and False Negative Rate (FNR) for males and females.

Definitions

False Positive Rate (FPR):

FPR =FP /(FP+TN)

The percentage of people who did NOT have high income but were incorrectly predicted to have high income.

False Negative Rate (FNR):

FNR= FN /(FN+TP)

The percentage of people who DID have high income but were incorrectly predicted to have low income.

```
from sklearn.metrics import confusion_matrix

def get_rates(model, X_test, y_test, subgroup_column, subgroup_value):
    # 1. Filter subgroup
    subgroup_mask = X_test[subgroup_column] == subgroup_value
    X_subgroup = X_test[subgroup_mask]
    y_subgroup = y_test[subgroup_mask]

    # 2. Predictions
    y_pred_subgroup = model.predict(X_subgroup)

    # 3. Confusion matrix
    tn, fp, fn, tp = confusion_matrix(y_subgroup, y_pred_subgroup).ravel()

    # 4. Rates
    fpr = fp / (fp + tn)
    fnr = fn / (fn + tp)

    return fpr, fnr


# 1. Calculate the rates for males and females
fpr_male, fnr_male = get_rates(model, X_test, y_test, 'sex', 'Male')
fpr_female, fnr_female = get_rates(model, X_test, y_test, 'sex', 'Female')

# 2. Print results
print(f"Male - False Positive Rate: {fpr_male:.2%}, False Negative Rate: {fnr_male:.2%}")
print(f"Female - False Positive Rate: {fpr_female:.2%}, False Negative Rate: {fnr_female:.2%}")

# 1. Calculate the rates for males and females
fpr_male, fnr_male = get_rates(model, X_test, y_test, 'sex', 'Male')
fpr_female, fnr_female = get_rates(model, X_test, y_test, 'sex', 'Female')

# 2. Print results
print(f"Male - False Positive Rate: {fpr_male:.2%}, False Negative Rate: {fnr_male:.2%}")
print(f"Female - False Positive Rate: {fpr_female:.2%}, False Negative Rate: {fnr_female:.2%}")
```

```
Male - False Positive Rate: 10.26%, False Negative Rate: 37.80%
Female - False Positive Rate: 2.81%, False Negative Rate: 47.84%
Male - False Positive Rate: 10.26%, False Negative Rate: 37.80%
Female - False Positive Rate: 2.81%, False Negative Rate: 47.84%
```

```
print("Male Accuracy:", acc_male)
print("Female Accuracy:", acc_female)

print("Male FPR:", fpr_male)
print("Male FNR:", fnr_male)

print("Female FPR:", fpr_female)
print("Female FNR:", fnr_female)
```

```
Male Accuracy: 0.81201171875
Female Accuracy: 0.9180722891566265
Male FPR: 0.10261854210898796
Male FNR: 0.3779527559055118
Female FPR: 0.028146989835809225
Female FNR: 0.4783861671469741
```

Reflective Knowledge Check

1. Analyze Your Results

From the subgroup accuracy results:

Accuracy for Males: 81.20%

Accuracy for Females: 91.81%

There is / is not a significant difference in performance between the two groups.

The model performs better for [Females], as indicated by the higher accuracy. This suggests that the model may be biased toward that group, since it predicts outcomes more correctly for them.

2. Interpret the Errors

From the calculated rates:

Male FPR: [10.26%], Male FNR: [37.80%] Female FPR: [2.81%], Female FNR: [47.84%]

The model is more likely to make a False Positive error for [Males], since that group has a higher FPR.

Real-world consequence (loan example): A False Positive means predicting high income when the person actually has low income.

This could lead to approving a loan for someone who may not be able to repay it Financial institutions could face higher default risk The model may unfairly favor one group in loan approvals 3. Justify a Decision (Hiring Scenario)

In a hiring context:

False Positive (FPR): Hiring someone predicted to be high-income potential when they are not False Negative (FNR): Rejecting someone who actually has high potential

More harmful error:

Typically, FNR is more harmful, because it denies qualified candidates opportunities This can lead to systematic exclusion of a group

Based on the results:

If one group has a higher FNR, they are unfairly being rejected more often If one group has a higher FPR, they are being unfairly favored

Decision:

I would [not approve] this model for deployment

Justification:

The model shows disparate impact, as error rates differ between groups This indicates potential bias, making it unsuitable for fair hiring decisions 4. Propose a Mitigation

Removing the 'sex' column will not fully eliminate bias.

Why?

Other features such as:

occupation education hours-per-week marital-status

may still be correlated with gender

This is called:

Proxy bias (indirect discrimination) Conclusion:

Even after removing 'sex', the model may still learn gender-related patterns True fairness requires: Feature analysis Bias mitigation techniques (reweighting, fairness constraints, etc.)